

Projeto 12- JornalIFPB

Descrição Geral: É uma plataforma digital voltada à divulgação de notícias, oportunidades e informações institucionais do Instituto Federal da Paraíba (IFPB). Aproximar a comunidade acadêmica — estudantes, professores e servidores — de conteúdos atualizados sobre eventos, vagas de estágio, bolsas e editais. O sistema parte do problema de dificuldade no acesso centralizado a informações acadêmicas e oportunidades, oferecendo uma solução que unifica esses dados em um único portal digital. O projeto estimula o uso da tecnologia como ferramenta de comunicação, transparência e integração dentro do ambiente educacional, reforçando o papel social e informativo do IFPB.

Objetivo do Miniprojeto: Transformar o site estático do JornalIFPB em uma aplicação interativa utilizando JavaScript puro. O sistema deve permitir manipular dados dinamicamente e criar interações reais com o usuário, tornando o conteúdo de notícias, vagas e editais dinâmico e filtrável em tempo real.

Conceitos de JavaScript a serem explorados:

- Vetores e Objetos (armazenamento e manipulação de dados)
 - Manipulação da DOM (createElement, innerHTML, appendChild)
 - Eventos (click, submit, input, change)
 - Funções de alta ordem: map(), filter(), forEach(), find(), reduce() entre outros
 - Estruturas condicionais e loops (if/else, for, while, switch)
 - Armazenamento local (localStorage)
 - Modularização e funções reutilizáveis
-

Funcionalidades Detalhadas:

1. Estrutura de Dados Inicial:

Crie arrays de objetos representando os elementos principais do sistema, como **notícias**, **vagas** e **editais**.

Cada objeto deve conter campos como id, titulo, categoria, data, descricao, e curtidas.

Exemplo:

```
const noticias = [
```

```
{ id: 1, titulo: 'Semana de Tecnologia do IFPB', categoria: 'Eventos', data: '2025-10-20',  
curtidas: 0 },
```

```
{ id: 2, titulo: 'Novas bolsas de pesquisa abertas', categoria: 'Oportunidades', data: '2025-10-  
18', curtidas: 3 }
```

```
];
```

2. **Renderização Dinâmica:**

Percorra os arrays e gere elementos HTML dinamicamente (cards, listas ou tabelas).
Use `createElement()` ou `innerHTML` para exibir notícias, vagas e editais de forma automatizada.
Cada card deve conter **botões de ação**, como “Curtir”, “Ver mais” ou “Salvar”.

3. **Filtros e Pesquisa:**

Implemente campos de busca e filtros nos módulos de **vagas** e **editais**.
Use `filter()` e `includes()` para exibir apenas os resultados correspondentes.
Exemplo de filtros:

- Tipo de edital (Pesquisa, Extensão, Bolsa)
 - Área da vaga (Informática, Administração, Eletrônica etc.)
-

4. **Formulário de Cadastro:**

Adicione um formulário para **inserir novas notícias ou vagas**.
Valide entradas antes de adicionar ao array.
Use `preventDefault()` para evitar recarregamento e exiba mensagem de confirmação após o cadastro.

5. **Persistência Local:**

Salve os dados no `localStorage` usando `JSON.stringify()`.
Recupere com `JSON.parse()` ao recarregar a página, garantindo que as informações sejam mantidas.

6. **Feedback e Interatividade:**

Mostre mensagens no DOM (alertas ou modais) para ações bem-sucedidas, como “Vaga adicionada com sucesso” ou “Edital salvo nos favoritos”.
Permita curtir, favoritar ou marcar itens como lidos, atualizando os contadores com `sort()` e `reduce()`.

7. Ordenação e Ranking:

Implemente botões para ordenar os registros (por título, data ou curtidas).
A listagem deve se atualizar automaticamente após a ordenação.

• Validações:

Garanta que todos os campos do formulário possuam **valores válidos e mínimos**.
Exemplo: Título com pelo menos 3 caracteres. Data válida (no formato ISO). Categoria selecionada. Exiba mensagens visuais de erro no DOM.

Estrutura HTML Sugerida (Explicação):

O HTML base deve conter as seções principais do site.
O aluno deve expandir a estrutura conforme o projeto evolui.

Sugere-se:

- <header> — título e menu principal.
- <section id='filtros'> — campos de busca e botões de filtro.
- <section id='conteudo'> — área dinâmica para cards de notícias, vagas ou editais.
- <form id='cadastro'> — formulário para adicionar novos registros.
- <footer> — créditos e informações adicionais.

Os elementos devem possuir **classes e IDs** apropriados para facilitar a manipulação da DOM.
Itens como , <div> e <button> devem ser **gerados dinamicamente** para praticar manipulação com JavaScript.

Dicas de Implementação:

1. Defina os arrays de dados iniciais e as funções de renderização.
2. Desenvolva o sistema de busca e filtros.
3. Crie o formulário de cadastro e os eventos de submit.
4. Adicione salvamento e recuperação de dados com localStorage.
5. Insira feedback visual (mensagens, animações simples).
6. Teste e refine o layout, garantindo responsividade.

Extras Opcionais:

- Adicionar **filtro por data de publicação**.
- Implementar **favoritos** salvos localmente.
- Criar **modal de detalhes** para notícias e editais.
- Exportar/importar dados em formato JSON.